

SCC Factor-Graph Propagation

A Hybrid Inference Algorithm for Logical Credal Networks

IBM Research

Abstract

We describe **SCC Factor-Graph Propagation (SCC-FGP)**, a hybrid approximate-inference algorithm for Logical Credal Networks (LCNs) that computes lower/upper marginal probability bounds for all propositions. The algorithm first decomposes the LCN’s *structure graph* into its **Strongly Connected Components** (SCCs) using Tarjan’s linear-time algorithm, contracting every cycle into a single *super-node*. The result is a *condensation graph*—a directed acyclic graph (DAG)—over which we build a **factor graph** whose factor nodes are the super-nodes and whose variable nodes are the *interface variables* shared between super-nodes. Belief propagation then runs *exactly* on this acyclic factor graph: messages are *credal sets* (probability intervals on the interface variables) and each factor-to-variable message is obtained by solving a local non-linear program (NLP) over the joint space of one super-node and its interface variables. Because all cycles are isolated *inside* the super-nodes, the global propagation is finite and exact (two passes for a tree; one forward and one backward pass along a topological order for a general DAG); the only source of approximation is the local NLP solved inside a non-trivial (cyclic) super-node, which itself may be solved exactly (e.g. **ipopt**) or approximately (e.g. ARIEL). We give the full algorithm, complete pseudocode, and an end-to-end worked example on a 7-variable LCN (A, B, C, D, E, X, Y) with *concrete numeric bounds* in which we exhibit *every* message, the exact NLP that produces it, and its computed interval; the resulting marginals are verified to match the exact 128-world global NLP to four decimals. We close with a point-by-point comparison against the ARIEL message-passing scheme, illustrated on the same example.

Contents

1	Background: Logical Credal Networks	2
1.1	Semantics: the credal set	2
1.2	Marginal inference	3
1.3	The structure graph	3
2	From Cycles to a DAG: SCC Decomposition	3
2.1	Strongly connected components	3
2.2	The condensation graph	4
2.3	Interface variables	4
3	The Condensation Factor Graph	4
3.1	Messages are credal sets	4
3.2	Final belief	5
4	The SCC-FGP Algorithm	6
4.1	Overview	6
4.2	Pseudocode	7

5	Worked Example: a 7-Variable LCN	8
5.1	The LCN	8
5.2	SCC decomposition	8
5.3	The condensation factor graph	9
5.4	Pass 1 — Collect (toward root $[S_E]$)	9
5.5	Pass 2 — Distribute (away from root $[S_E]$)	10
5.6	Stage 4 — Final marginals	12
5.7	Summary of all messages	13
5.8	What the example demonstrates	13
6	Comparison with the ARIEL Scheme	14
6.1	Critical Difference 1 — the graph representation	14
6.2	Critical Difference 2 — handling of cycles	14
6.3	Critical Difference 3 — the NLPs	14
6.4	Side-by-side trace on the example	14
6.5	Summary table	15
6.6	A hybrid relationship	15
7	Complexity and Practical Notes	16

1 Background: Logical Credal Networks

A **Logical Credal Network** (LCN) is a probabilistic logic model that combines propositional logic with imprecise (interval) probability bounds. An LCN program $\mathcal{L}$ is a finite set of labelled **sentences**, each of one of two forms:

$$\text{(Type 1)} \quad l \leq P(\phi) \leq u, \quad (1)$$

$$\text{(Type 2)} \quad l \leq P(\phi \mid \psi) \leq u, \quad (2)$$

where ϕ, ψ are propositional formulas over a set of Boolean atoms (propositions) $\mathbf{X} = \{X_1, \dots, X_n\}$ built from the connectives **and**, **or**, **xor**, **nand**, **not**, and $0 \leq l \leq u \leq 1$. Each sentence additionally carries a flag τ that governs how it contributes to the structure graph (Section 1.3).

1.1 Semantics: the credal set

An LCN does *not* define a single joint distribution. Instead it defines a **credal set** $\mathcal{K}(\mathbf{X})$: the (convex) set of all joint probability distributions P over the 2^n truth assignments (“worlds”) $\mathbf{x} \in \{0, 1\}^n$ that satisfy

1. the **probability axioms** $p(\mathbf{x}) \geq 0$, $\sum_{\mathbf{x}} p(\mathbf{x}) = 1$;
2. every **probabilistic constraint** (1)–(2), where each probability is the sum of the world-probabilities consistent with the formula:

$$P(\phi) = \sum_{\mathbf{x} \models \phi} p(\mathbf{x}), \quad P(\phi \mid \psi) = \frac{P(\phi \wedge \psi)}{P(\psi)};$$

3. the **Local Markov Condition (LMC)** independence constraints derived from the structure graph (below), each of which translates to a *quadratic equality* of the form $P(\mathbf{x}, \mathbf{y}, \mathbf{z}) P(\mathbf{z}) = P(\mathbf{x}, \mathbf{z}) P(\mathbf{y}, \mathbf{z})$ whenever $\mathbf{X} \perp \mathbf{Y} \mid \mathbf{Z}$.

A Type-2 sentence $l \leq P(\phi \mid \psi) \leq u$ is *linearized* into two homogeneous linear inequalities over the world-probabilities,

$$l \cdot P(\psi) \leq P(\phi \wedge \psi) \leq u \cdot P(\psi), \quad (3)$$

which become *non-linear* only when $P(\psi)$ is itself an optimization variable (a sum of $p(\mathbf{x})$). This is the sole source of non-convexity in the feasible region.

1.2 Marginal inference

Given a query proposition Q , marginal inference computes the tightest bounds

$$\underline{P}(Q) = \min_{P \in \mathcal{K}(\mathbf{X})} \sum_{\mathbf{x} \models Q} p(\mathbf{x}), \quad \overline{P}(Q) = \max_{P \in \mathcal{K}(\mathbf{X})} \sum_{\mathbf{x} \models Q} p(\mathbf{x}). \quad (4)$$

Solving (4) over the full credal set is a single global NLP with 2^n variables; it is exact but intractable (exponential variables, non-convex) for all but the smallest LCNs. SCC-FGP is a *structured* approximation of this global problem.

1.3 The structure graph

The structure graph $\mathcal{G}_{\text{struct}}$ is a *mixed* graph (directed + undirected edges) over the atoms, built from the sentences by the following rules (implemented in `LCN.build_structure_graph`):

1. **Undirected edges (intra-formula).** For each sentence with $\tau = \text{True}$, add an undirected edge between every pair of atoms occurring in the *conditioned* formula ϕ .
2. **Directed edges (conditioning).** For each Type-2 sentence $P(\phi \mid \psi)$, add a directed edge $u \rightarrow v$ from every atom $u \in \psi$ to every atom $v \in \phi$.
3. **Bidirected collapse.** If both $u \rightarrow v$ and $v \rightarrow u$ are present, replace them by a single undirected edge $u - v$.
4. **Cleanup.** Remove formula-nodes and duplicate edges.

Edges of $\mathcal{G}_{\text{struct}}$ encode probabilistic dependence. A directed cycle in $\mathcal{G}_{\text{struct}}$ marks a set of mutually-dependent atoms that *cannot* be ordered acyclically—these are exactly the cycles that make loopy propagation schemes (Section 6) inexact.

2 From Cycles to a DAG: SCC Decomposition

2.1 Strongly connected components

Definition 1 (SCC). A ***strongly connected component*** of a directed graph $\mathcal{G}$ is a maximal set of vertices S such that for every pair $u, v \in S$ there is a directed path $u \rightsquigarrow v$ and a directed path $v \rightsquigarrow u$ in $\mathcal{G}$. A vertex with no cycle through it forms a trivial (singleton) SCC.

To apply SCC analysis we first orient $\mathcal{G}_{\text{struct}}$: an undirected edge $u - v$ (which records a symmetric / cyclic dependence) is treated as the pair of directed edges $u \rightarrow v$ and $v \rightarrow u$, so that any atoms tied by undirected edges automatically land in the same SCC. (Equivalently, `LCN.simplify_structure_graph` already contracts each undirected clique into one meta-node; the SCC step then operates on the remaining directed graph and merges any further directed cycles.)

Proposition 1 (Tarjan). The SCCs of a directed graph $\mathcal{G} = (V, E)$ can be computed in $O(|V| + |E|)$ time, and they can be emitted in reverse topological order of the condensation graph.

2.2 The condensation graph

Definition 2 (Condensation / super-node). *Contracting each SCC S_j to a single **super-node** and keeping one edge $S_i \rightarrow S_j$ whenever $\mathcal{G}_{\text{struct}}$ has an edge from some atom in S_i to some atom in S_j (with $i \neq j$) yields the **condensation graph** $\mathcal{G}_{\text{sc}}$. $\mathcal{G}_{\text{sc}}$ is always a directed acyclic graph (DAG): any cycle among super-nodes would contradict the maximality of SCCs.*

We write $\text{Vars}(S_j)$ for the atoms inside super-node S_j . A super-node is **trivial** if $|\text{Vars}(S_j)| = 1$ and the atom has no self-cycle, and **cyclic** (non-trivial) otherwise. The whole point of the decomposition is captured by:

Every directed cycle of the original network lives entirely inside some super-node. The graph between super-nodes is acyclic.

2.3 Interface variables

Definition 3 (Interface variable). *An atom V is an **interface variable** of super-node S_j if V participates in an edge of $\mathcal{G}_{\text{sc}}$ incident to S_j , i.e. V is the tail of a linking constraint $S_j \rightarrow S_c$ (an output interface toward child S_c) or the head/parent atom referenced by an incoming linking constraint $S_p \rightarrow S_j$. Interface variables are the only atoms whose beliefs are exchanged between super-nodes; the remaining atoms are **internal** (hidden) to their super-node.*

3 The Condensation Factor Graph

SCC-FGP runs belief propagation not on $\mathcal{G}_{\text{sc}}$ directly but on a **factor graph** $\mathcal{F}$ derived from it—a bipartite graph with two node types:

- **Factor nodes** $[S_j]$ (squares): one per super-node. The factor $[S_j]$ *owns* (i) the internal probabilistic constraints of S_j , (ii) the LMC quadratic constraints local to $\text{Vars}(S_j)$, and (iii) the *linking constraints* $l \leq P(\phi \mid \psi) \leq u$ that connect S_j to a neighbour.
- **Variable nodes** (V) (circles): one per interface variable V . A variable node holds the current *belief* (credal set / interval) about V .

Edges connect a factor $[S_j]$ to a variable (V) iff V is an interface variable of S_j . Because $\mathcal{G}_{\text{sc}}$ is a DAG and each interface variable links exactly the super-nodes that mention it, $\mathcal{F}$ inherits an acyclic (tree- or polytree-shaped) topology at the level of super-nodes.

3.1 Messages are credal sets

Definition 4 (Message). *A **message** on edge $(V) - [S_j]$ is a credal set over the single interface variable V . For a Boolean V this is simply an interval $\mathcal{K}(V) = [l_V, u_V]$ bounding $P(V=1)$ (with $P(V=0) \in [1-u_V, 1-l_V]$). In general it is the marginal credal set $\mathcal{K}(V)$, a convex polytope; for multi-state interfaces it is a vector of per-state intervals (or a polytope given by linear inequalities, e.g. from Fourier–Motzkin elimination).*

There are two message types, with the standard belief-propagation roles.

Variable-to-factor message $\mu(V \rightarrow F)$. The belief about V assembled from *every neighbour* of V *except* F :

$$\mu(V \rightarrow F) = \bigcap_{F' \in \mathcal{N}(V) \setminus \{F\}} \mu(F' \rightarrow V). \quad (5)$$

“ $\cap$ ” is credal-set *intersection*; for intervals it is

$$\mu(V \rightarrow F) = \left[\max_{F'} l_{F' \rightarrow V}, \min_{F'} u_{F' \rightarrow V} \right].$$

A leaf variable (one neighbour) sends a *vacuous* message $[0, 1]$ if it has received nothing, or simply forwards its single incoming message.

Factor-to-variable message $\mu(F \rightarrow V)$. The core computation. Factor $F = [S_j]$ tells V what its own constraints, plus the messages from its *other* interface variables, imply about V . This requires solving a **local NLP** over the joint space of $\text{Vars}(S_j)$ together with the interface atoms of S_j :

$$\begin{aligned} \mu(F \rightarrow V) : \quad & \underline{P}(V) = \min_{\mathbf{p}} P(V=1), \quad \bar{P}(V) = \max_{\mathbf{p}} P(V=1), \\ \text{s.t.} \quad & \sum_{\mathbf{w}} p(\mathbf{w}) = 1, \quad p(\mathbf{w}) \geq 0, \\ & \text{internal} + \text{LMC} + \text{linking constraints of } S_j, \\ & \text{message constraints } \mu(V' \rightarrow F) \text{ for every other interface } V' \neq V. \end{aligned} \quad (6)$$

Here $\mathbf{p} = \{p(\mathbf{w})\}$ ranges over the joint distribution of S_j ’s atoms and its interface atoms, and “ $V=1$ ” is the union of worlds $\mathbf{w}$ in which V is true. Incoming interval messages $\mu(V' \rightarrow F) = [l', u']$ enter as the two linear inequalities $l' \leq P(V'=1) \leq u'$.

Why a ratio sometimes appears (LP vs. LFP). When the message target is a *marginal* $P(V)$, both objective and constraints are linear in $\mathbf{p}$ *except* for the conditional (linking / internal) constraints, which after the linearization (3) are homogeneous linear inequalities—so a marginal message is computed by an **LP** when no internal LMC quadratics are active, and by an **NLP** when they are. If instead one needs bounds on a *conditional* $P(Q \mid E) = P(Q, E)/P(E)$, the objective is a ratio of linear forms—a **Linear-Fractional Program (LFP)**—which is solved by the **Charnes–Cooper transformation** (substitute $t = 1/P(E)$, $y(\mathbf{w}) = t p(\mathbf{w})$) into an equivalent LP.

3.2 Final belief

After messages have flowed both ways, the final marginal credal set of an interface variable V is the intersection of *all* messages arriving at its variable node:

$$\mathcal{K}(V) = \bigcap_{F \in \mathcal{N}(V)} \mu(F \rightarrow V) = \left[\max_F l_{F \rightarrow V}, \min_F u_{F \rightarrow V} \right]. \quad (7)$$

For an **internal** atom W (hidden inside S_j , never on an edge of $\mathcal{F}$), we run one final NLP on $[S_j]$ constrained by *all* final incoming messages and optimize $P(W=1)$:

$$\mathcal{K}(W) = \left[\min_{\mathbf{p}} P(W=1), \max_{\mathbf{p}} P(W=1) \right] \text{ s.t. internal/LMC/linking constraints} + \bigcup_{V \in \text{Int}(S_j)} \mu(V \rightarrow F). \quad (8)$$

4 The SCC-FGP Algorithm

4.1 Overview

The algorithm has four stages:

1. **Decompose.** Build $\mathcal{G}_{\text{struct}}$, run Tarjan to get SCCs, contract into the condensation DAG $\mathcal{G}_{\text{sc}}$, identify interface variables, and build the factor graph $\mathcal{F}$.
2. **Collect (forward).** Process factors in a topological order; each factor, having received messages from its already-processed neighbours, emits factor-to-variable messages toward its children.
3. **Distribute (backward).** Process factors in reverse topological order; each factor emits factor-to-variable messages back toward its parents.
4. **Extract.** Compute final beliefs: intersect messages at each interface variable (7); solve one local NLP per super-node for its internal atoms (8).

For a *tree*-shaped factor graph this is exactly two passes (collect-to-root, distribute-from-root). For a general DAG of super-nodes the forward pass follows a topological sort and the backward pass its reverse; each factor combines *all* incoming messages from already-processed neighbours before emitting, which is the generalized (non-loopy) belief propagation rule.

4.2 Pseudocode

Algorithm 1 SCC Factor-Graph Propagation (SCC-FGP)

Require: LCN $\mathcal{L}$; local-solver SOLVE (exact `ipopt` or `ARIEL`)

Ensure: Bounds $[\underline{P}(W), \overline{P}(W)]$ for every atom W

▷ STAGE 1 — DECOMPOSE

- 1: $\mathcal{G}_{\text{struct}} \leftarrow \text{BUILDSTRUCTUREGRAPH}(\mathcal{L})$
- 2: Orient undirected edges as symmetric directed pairs
- 3: $\{S_1, \dots, S_k\} \leftarrow \text{TARJAN}(\mathcal{G}_{\text{struct}})$ ▷ SCCs, $O(|V|+|E|)$
- 4: $\mathcal{G}_{\text{scc}} \leftarrow \text{CONDENSE}(\mathcal{G}_{\text{struct}}, \{S_j\})$ ▷ DAG of super-nodes
- 5: Identify interface variables; build factor graph $\mathcal{F}$
- 6: $\mathbf{o} \leftarrow \text{TOPOLOGICALSORT}(\mathcal{G}_{\text{scc}})$

▷ STAGE 2 — COLLECT (FORWARD, ORDER $\mathbf{o}$)

- 7: **for** each factor $[S_i]$ in order $\mathbf{o}$ **do**
- 8: Gather incoming $\mu(V \rightarrow [S_i])$ from already-processed neighbours ▷ Eq. (5)
- 9: **for** each child interface V of $[S_i]$ **do**
- 10: $\mu(S_i \rightarrow V) \leftarrow \text{SOLVE}(\text{local NLP (6) for } V)$
- 11: Push $\mu(S_i \rightarrow V)$ onto variable node (V)
- 12: **end for**
- 13: **end for**

▷ STAGE 3 — DISTRIBUTE (BACKWARD, ORDER REVERSE $\mathbf{o}$)

- 14: **for** each factor $[S_i]$ in reverse order $\mathbf{o}$ **do**
- 15: Gather incoming $\mu(V \rightarrow [S_i])$ from all neighbours except the target ▷ Eq. (5)
- 16: **for** each parent interface V of $[S_i]$ **do**
- 17: $\mu(S_i \rightarrow V) \leftarrow \text{SOLVE}(\text{local NLP (6) for } V)$
- 18: Push $\mu(S_i \rightarrow V)$ onto variable node (V)
- 19: **end for**
- 20: **end for**

▷ STAGE 4 — EXTRACT MARGINALS

- 21: **for** each interface variable V **do**
- 22: $\mathcal{K}(V) \leftarrow [\max_F l_{F \rightarrow V}, \min_F u_{F \rightarrow V}]$ ▷ Eq. (7)
- 23: **end for**
- 24: **for** each super-node S_j **do**
- 25: **for** each internal atom $W \in \text{Vars}(S_j)$ **do**
- 26: $\mathcal{K}(W) \leftarrow \text{SOLVE}(\text{final NLP (8) for } W)$
- 27: **end for**
- 28: **end for**
- 29: **return** $\{[\underline{P}(W), \overline{P}(W)]\}_{W \in \mathbf{x}}$

Theorem 1 (Finiteness and global exactness). *On the acyclic factor graph $\mathcal{F}$, SCC-FGP terminates after exactly one forward and one backward pass over the k super-nodes, computing $O(|E(\mathcal{F})|)$ messages. If every local NLP (6) is solved to global optimality, the propagation introduces no additional looseness beyond the structured (per-super-node marginalization) approximation: there is no double-counting of evidence, because no message ever traverses a cycle at the super-node level.*

Proposition 2 (Sole source of approximation). *The only approximation in SCC-FGP arises inside a cyclic super-node whose local NLP (6) is too large to solve exactly and is instead handed to an approximate solver (e.g. `ARIEL`). Trivial (acyclic) super-nodes yield LPs/LFPs solved exactly.*

5 Worked Example: a 7-Variable LCN

We now make every step concrete on a 7-proposition LCN whose condensation graph is a non-trivial (non-tree) DAG, so that the general multi-parent / multi-child message rules are exercised.

5.1 The LCN

Propositions (7): $\{X, Y, A, B, C, D, E\}$.

Constraints (with concrete numeric bounds):

Label	Sentence	Numeric form	Role
c_X	$P(X)$	$\in [0.60, 0.80]$	root prior
c_Y	$P(Y)$	$\in [0.30, 0.50]$	root prior
c_{XA}	$P(A \mid X)$	$\in [0.70, 0.90]$	link $X \rightarrow A$
c_{YB}	$P(B \mid Y)$	$\in [0.20, 0.40]$	link $Y \rightarrow B$
c_{AB}	$P(B \mid A)$	$\in [0.50, 0.70]$	link $A \rightarrow B$
c_{BC}	$P(C \mid B)$	$\in [0.60, 0.80]$	<i>cycle</i> edge
c_{CD}	$P(D \mid C)$	$\in [0.40, 0.60]$	<i>cycle</i> edge
c_{DB}	$P(B \mid D)$	$\in [0.30, 0.50]$	<i>cycle</i> edge
c_{CE}	$P(E \mid C)$	$\in [0.80, 0.90]$	link $C \rightarrow E$
$c_{\bar{C}E}$	$P(E \mid \neg C)$	$\in [0.10, 0.30]$	link $C \rightarrow E$
c_E	$P(E)$	$\in [0.50, 0.70]$	leaf prior

The last two sentences, $c_{\bar{C}E}$ and c_E , are added so that the *backward* pass carries real information: c_E pins the marginal of the leaf E , and together with $c_{CE}, c_{\bar{C}E}$ this constrains $P(C)$ when propagated back up. (Without them, every backward message in this all-“downward” network would be the vacuous interval $[0, 1]$.)

LCN conditional semantics. A Type-2 sentence $l \leq P(H \mid G) \leq u$ constrains *only* the conditioning event $G=\text{true}$; via (3) it becomes the single pair of inequalities $l P(G=1) \leq P(H=1, G=1) \leq u P(G=1)$. (The reverse case $G=\text{false}$ is governed by a *separate* sentence, e.g. $c_{\bar{C}E}$ covers $E \mid \neg C$.) This matches the encoding in `lcn/inference/marginal/exact.py`.

Structure graph. The directed edges are $X \rightarrow A$, $Y \rightarrow B$, $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, $D \rightarrow B$, $C \rightarrow E$ (the formula $\neg C$ in $c_{\bar{C}E}$ contributes the same edge $C \rightarrow E$). The three constraints c_{BC}, c_{CD}, c_{DB} form a directed cycle $B \rightarrow C \rightarrow D \rightarrow B$.

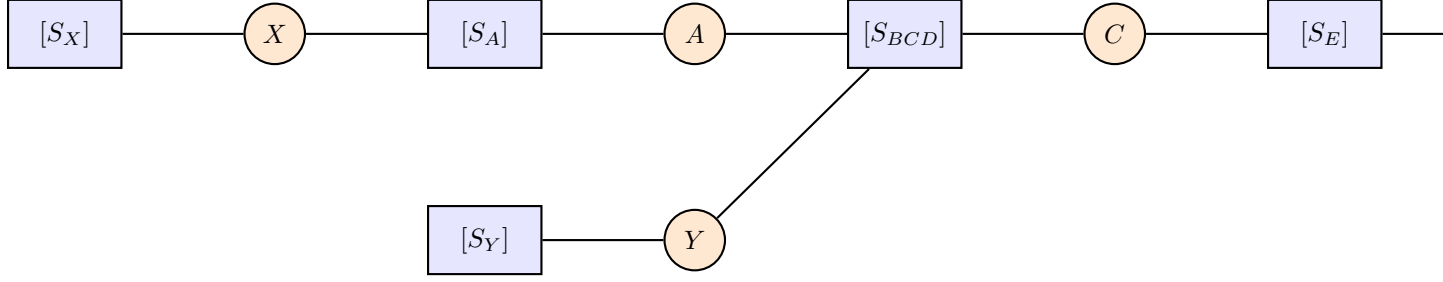
5.2 SCC decomposition

Tarjan’s algorithm finds one non-trivial SCC, $S_{BCD} = \{B, C, D\}$ (the cycle), and four trivial SCCs $S_X = \{X\}$, $S_Y = \{Y\}$, $S_A = \{A\}$, $S_E = \{E\}$. The condensation graph $\mathcal{G}_{\text{sc}}$ is the DAG

$$S_X \rightarrow S_A \rightarrow S_{BCD}, \quad S_Y \rightarrow S_{BCD}, \quad S_{BCD} \rightarrow S_E.$$

A valid topological order is $\mathbf{o} = (S_X, S_Y, S_A, S_{BCD}, S_E)$. The **interface variables** are X (links $S_X - S_A$), A (links $S_A - S_{BCD}$), Y (links $S_Y - S_{BCD}$), C (links $S_{BCD} - S_E$), and E (output of S_E). The atoms B and D are **internal** to S_{BCD} .

5.3 The condensation factor graph



Squares are factor nodes (super-nodes), circles are interface variable nodes. Edges: $(X) - [S_X], [S_A]$; $(A) - [S_A], [S_{BCD}]$; $(Y) - [S_Y], [S_{BCD}]$; $(C) - [S_{BCD}], [S_E]$; $(E) - [S_E]$. The factor graph is a **tree** with hub $[S_{BCD}]$; we root it at $[S_E]$ and run two passes.

Notation for the worked NLPs. Inside a factor we index joint probabilities by the truth values of the relevant atoms; e.g. in $[S_A]$, $p_{ax} = P(A=a, X=x)$; in $[S_E]$, $p_{ec} = P(E=e, C=c)$; in $[S_{BCD}]$, $p_{bcdy} = P(B=b, C=c, D=d, A=a, Y=y)$ ($2^5 = 32$ variables). A linking constraint such as $c_{AB} : 0.50 \leq P(B | A) \leq 0.70$ expands (via (3), for the conditioning event $A=1$ *only*) to the single pair $0.50 P(A=1) \leq P(B=1, A=1) \leq 0.70 P(A=1)$, each side a sum of the p -variables. There is *no* companion constraint for $A=0$ unless a separate sentence (such as $c_{\bar{C}E}$ for $E | \neg C$) supplies one.

All intervals below are the actual optimal values produced by solving the stated LP/NLP for the numeric bounds of the constraint table. As a ground-truth check, the single global NLP over all $2^7 = 128$ worlds (Section 1) yields *identical* final marginals (Section 5.6, Table 1): on this network the condensation graph is a polytree and every SCC is solved exactly, so SCC-FGP is *exact* here.

5.4 Pass 1 — Collect (toward root $[S_E]$)

(1.1) $\mu(S_X \rightarrow X)$ — **LP**.

Variable: $p_x = P(X=1)$.

Constraints: $0.60 \leq p_x \leq 0.80$.

Objective: $\min / \max p_x$.

$$\Rightarrow \mu(S_X \rightarrow X) = \mathcal{K}(X) = [0.60, 0.80].$$

(1.2) $\mu(S_Y \rightarrow Y)$ — **LP**. Symmetric: variable p_y , constraint $0.30 \leq p_y \leq 0.50$, $\Rightarrow \mu(S_Y \rightarrow Y) = \mathcal{K}(Y) = [0.30, 0.50]$.

(1.3) $\mu(X \rightarrow S_A)$ **and** $\mu(Y \rightarrow S_{BCD})$ — **V→F**. (X) has one other neighbour, so it forwards its incoming message: $\mu(X \rightarrow S_A) = [0.60, 0.80]$. Likewise $\mu(Y \rightarrow S_{BCD}) = [0.30, 0.50]$.

(1.4) $\mu(S_A \rightarrow A)$ — **NLP over $\{A, X\}$.**

$$\begin{aligned}
&\text{Variables: } p_{ax}, a, x \in \{0, 1\} \quad (4 \text{ vars}). \\
&\text{Constraints: } \sum_{ax} p_{ax} = 1, \quad p_{ax} \geq 0; \\
&\quad \text{incoming } \mu(X \rightarrow S_A) : 0.60 \leq (p_{01} + p_{11}) \leq 0.80; \\
&\quad c_{XA} (X=1 \text{ only}) : 0.70 (p_{01} + p_{11}) \leq p_{11} \leq 0.90 (p_{01} + p_{11}). \\
&\text{Objective: } \min / \max P(A=1) = p_{10} + p_{11}. \\
&\Rightarrow \mu(S_A \rightarrow A) = \mathcal{K}_{\text{fwd}}(A) = [0.4200, 0.9400].
\end{aligned}$$

(*Why so wide?* c_{XA} pins A only through $X=1$; the mass on $X=0$ leaves $P(A=1 \mid X=0)$ unconstrained, so the lower bound $0.70 \cdot 0.60 = 0.42$ and upper bound $0.90 \cdot 0.60 + 0.40 = 0.94$ follow.)

(1.5) $\mu(A \rightarrow S_{BCD})$ — **V $\rightarrow$ F.** (A) forwards: $\mu(A \rightarrow S_{BCD}) = \mathcal{K}_{\text{fwd}}(A) = [0.4200, 0.9400]$.

(1.6) $\mu(S_{BCD} \rightarrow C)$ — **the large NLP over $\{B, C, D, A, Y\}$.** This is the heart of the example: it isolates the cycle. Every conditional below is enforced for its conditioning event = true *only*.

$$\begin{aligned}
&\text{Variables: } p_{bcday}, b, c, d, a, y \in \{0, 1\} \quad (32 \text{ vars}). \\
&\text{Constraints: } \sum p_{bcday} = 1, \quad p_{bcday} \geq 0; \\
&\quad (\text{incoming}) 0.4200 \leq P(A=1) = \sum_{b,c,d,y} p_{bcd1y} \leq 0.9400; \\
&\quad 0.30 \leq P(Y=1) = \sum_{b,c,d,a} p_{bcd a1} \leq 0.50; \\
&\quad (\text{linking}) c_{AB} : 0.50 P(A=1) \leq P(B=1, A=1) \leq 0.70 P(A=1); \\
&\quad c_{YB} : 0.20 P(Y=1) \leq P(B=1, Y=1) \leq 0.40 P(Y=1); \\
&\quad (\text{internal, cyclic}) c_{BC} : 0.60 P(B=1) \leq P(C=1, B=1) \leq 0.80 P(B=1); \\
&\quad c_{CD} : 0.40 P(C=1) \leq P(D=1, C=1) \leq 0.60 P(C=1); \\
&\quad c_{DB} : 0.30 P(D=1) \leq P(B=1, D=1) \leq 0.50 P(D=1). \\
&\text{Objective: } \min / \max P(C=1) = \sum_{a,b,d,y} p_{b1day}. \\
&\Rightarrow \mu(S_{BCD} \rightarrow C) = \mathcal{K}_{\text{fwd}}(C) = [0.1260, 0.9580].
\end{aligned}$$

The three constraints c_{BC}, c_{CD}, c_{DB} are mutually dependent (the cycle); solving this NLP *jointly* over all 32 worlds is precisely what captures the cyclic dependence exactly, instead of looping messages around it.

(1.7) $\mu(C \rightarrow S_E)$ — **V $\rightarrow$ F.** (C) forwards: $\mu(C \rightarrow S_E) = \mathcal{K}_{\text{fwd}}(C) = [0.1260, 0.9580]$. The root factor $[S_E]$ has now received all leaf-ward information.

5.5 Pass 2 — Distribute (away from root $[S_E]$)

(2.1) $\mu(S_E \rightarrow C)$ — **NLP over $\{E, C\}$.** Per the BP rule, the backward message out of $[S_E]$ to (C) uses $[S_E]$'s *other* inputs only and therefore *excludes* the message $\mu(C \rightarrow S_E)$ that just arrived

from C . What remains are $[S_E]$'s own constraints $c_{CE} (E | C)$, $c_{\bar{C}E} (E | \neg C)$ and the leaf prior $c_E (P(E))$. It is precisely c_E together with the two-sided $c_{CE}, c_{\bar{C}E}$ that lets E “speak back” about C .

$$\begin{aligned}
&\text{Variables: } p_{ec}, e, c \in \{0, 1\} \quad (4 \text{ vars}). \\
&\text{Constraints: } \sum_{ec} p_{ec} = 1, \quad p_{ec} \geq 0; \\
&\quad c_{CE} (C=1) : 0.80 (p_{01}+p_{11}) \leq p_{11} \leq 0.90 (p_{01}+p_{11}); \\
&\quad c_{\bar{C}E} (C=0) : 0.10 (p_{00}+p_{10}) \leq p_{10} \leq 0.30 (p_{00}+p_{10}); \\
&\quad c_E : 0.50 \leq P(E=1) = p_{10}+p_{11} \leq 0.70. \\
&\text{Objective: } \min / \max P(C=1) = p_{01}+p_{11}. \\
&\Rightarrow \mu(S_E \rightarrow C) = \mathcal{K}_{\text{bwd}}(C) = [0.3333, 0.8571].
\end{aligned}$$

Intuition: $P(E) = P(E | C)P(C) + P(E | \neg C)P(\neg C)$ with $P(E) \leq 0.70$ and $P(E | C) \geq 0.80$ forces $P(C) \leq 0.857$; with $P(E) \geq 0.50$ and $P(E | \neg C) \leq 0.30$ it forces $P(C) \geq 0.333$.

$$(2.2) \mu(C \rightarrow S_{BCD}) \text{ — } \mathbf{V} \rightarrow \mathbf{F}. \quad \mu(C \rightarrow S_{BCD}) = \mathcal{K}_{\text{bwd}}(C) = [0.3333, 0.8571].$$

(2.3) $\mu(S_{BCD} \rightarrow A)$ — **large NLP, objective on A** . Same 32-variable factor, but now the incoming messages are $\mu(Y \rightarrow S_{BCD}) = [0.30, 0.50]$ (forward) and $\mu(C \rightarrow S_{BCD}) = [0.3333, 0.8571]$ (backward); the message from A is *excluded*.

$$\begin{aligned}
&\text{Constraints: } \sum p_{bcdy} = 1, \quad p \geq 0; \quad 0.30 \leq P(Y=1) \leq 0.50; \\
&\quad 0.3333 \leq P(C=1) \leq 0.8571; \\
&\quad \text{linking } c_{YB}; \text{ internal } c_{BC}, c_{CD}, c_{DB} \text{ (cond. true only)}. \\
&\text{Objective: } \min / \max P(A=1) = \sum_{b,c,d,y} p_{bcd1y}. \\
&\Rightarrow \mu(S_{BCD} \rightarrow A) = \mathcal{K}_{\text{bwd}}(A) = [0.0000, 1.0000].
\end{aligned}$$

The result is *vacuous*: the only constraint touching A is $c_{AB} (B | A)$, which is excluded here (it lives on the A edge), and the internal constraints constrain B, C, D but impose no *lower* or *upper* bound back onto the marginal $P(A)$ given the available messages. Information in this network flows strictly downstream, so the backward message to a parent carries nothing.

(2.4) $\mu(S_{BCD} \rightarrow Y)$ — **large NLP, objective on Y** . Symmetric to (2.3): incoming $\mu(A \rightarrow S_{BCD}) = [0.4200, 0.9400]$ (forward) and $\mu(C \rightarrow S_{BCD}) = [0.3333, 0.8571]$ (backward), exclude Y ; linking c_{AB} ; internal c_{BC}, c_{CD}, c_{DB} ; objective $\min / \max P(Y=1)$ (through the excluded c_{YB}). $\Rightarrow \mu(S_{BCD} \rightarrow Y) = \mathcal{K}_{\text{bwd}}(Y) = [0.0000, 1.0000]$ — again vacuous, for the same reason.

$$(2.5) \mu(A \rightarrow S_A) \text{ — } \mathbf{V} \rightarrow \mathbf{F}. \quad \mu(A \rightarrow S_A) = \mathcal{K}_{\text{bwd}}(A) = [0.0000, 1.0000].$$

$$(2.6) \mu(S_A \rightarrow X) \text{ — } \mathbf{NLP} \text{ over } \{A, X\}.$$

$$\begin{aligned}
&\text{Variables: } p_{ax} \text{ (4 vars)}. \\
&\text{Constraints: } \sum p_{ax} = 1, \quad p \geq 0; \quad \mu(A \rightarrow S_A): 0.0 \leq P(A=1) = (p_{10}+p_{11}) \leq 1.0; \\
&\quad c_{XA} (X=1 \text{ only}). \\
&\text{Objective: } \min / \max P(X=1) = p_{01}+p_{11}. \\
&\Rightarrow \mu(S_A \rightarrow X) = \mathcal{K}_{\text{bwd}}(X) = [0.0000, 1.0000].
\end{aligned}$$

Vacuous, because the incoming backward message on A is itself vacuous and c_{XA} constrains $A \mid X$, not X .

(2.7) $\mu(Y \rightarrow S_Y) \text{ — } \mathbf{V} \rightarrow \mathbf{F}$. $\mu(Y \rightarrow S_Y) = \mathcal{K}_{\text{bwd}}(Y) = [0.0000, 1.0000]$ (used only if a final belief at Y is desired; $[S_Y]$ has no further constraints to add).

5.6 Stage 4 — Final marginals

Interface variables (intersection, Eq. (7)).

$$\begin{aligned}\mathcal{K}(X) &= \mu(S_X \rightarrow X) \cap \mu(S_A \rightarrow X) = [0.60, 0.80] \cap [0, 1] = [0.6000, 0.8000], \\ \mathcal{K}(A) &= \mu(S_A \rightarrow A) \cap \mu(S_{BCD} \rightarrow A) = [0.42, 0.94] \cap [0, 1] = [0.4200, 0.9400], \\ \mathcal{K}(Y) &= \mu(S_Y \rightarrow Y) \cap \mu(S_{BCD} \rightarrow Y) = [0.30, 0.50] \cap [0, 1] = [0.3000, 0.5000], \\ \mathcal{K}(C) &= \mu(S_{BCD} \rightarrow C) \cap \mu(S_E \rightarrow C) = [0.126, 0.958] \cap [0.333, 0.857] = [0.3333, 0.8571], \\ \mathcal{K}(E) &= \mu(S_E \rightarrow E) = [0.5000, 0.7000] \quad (\text{one NLP on } [S_E] \text{ with } c_{CE}, c_{\bar{C}E}, c_E).\end{aligned}$$

Only $\mathcal{K}(C)$ is tightened by the backward pass (the two messages genuinely intersect, $0.333 = \max(0.126, 0.333)$ and $0.857 = \min(0.958, 0.857)$); the other interfaces inherit their forward intervals because every backward message was vacuous.

Internal atoms B, D (Eq. (8)). Run one last NLP on $[S_{BCD}]$ (32 vars) constrained by *all* final incoming messages $\mu(A \rightarrow S_{BCD}) = [0.42, 0.94]$, $\mu(Y \rightarrow S_{BCD}) = [0.30, 0.50]$, $\mu(C \rightarrow S_{BCD}) = [0.3333, 0.8571]$, plus internal c_{BC}, c_{CD}, c_{DB} and linking c_{AB}, c_{YB} ; objectives $\min / \max P(B=1)$ and $\min / \max P(D=1)$:

$$\mathcal{K}(B) = [0.2100, 0.8200], \quad \mathcal{K}(D) = [0.1333, 0.8667].$$

Final marginals and exactness check. Table 1 lists the SCC-FGP marginals beside the bounds from the single global NLP over all $2^7 = 128$ worlds (Section 1). They are *identical* to four decimals: on this network the condensation graph is a polytree and every super-node NLP is solved exactly, so SCC-FGP attains the true tightest bounds.

Table 1: SCC-FGP marginals vs. the exact global NLP (all 128 worlds).

Variable	SCC-FGP $[\underline{P}, \bar{P}]$	Global (exact)	Match
A	[0.4200, 0.9400]	[0.4200, 0.9400]	✓
B	[0.2100, 0.8200]	[0.2100, 0.8200]	✓
C	[0.3333, 0.8571]	[0.3333, 0.8571]	✓
D	[0.1333, 0.8667]	[0.1333, 0.8667]	✓
E	[0.5000, 0.7000]	[0.5000, 0.7000]	✓
X	[0.6000, 0.8000]	[0.6000, 0.8000]	✓
Y	[0.3000, 0.5000]	[0.3000, 0.5000]	✓

5.7 Summary of all messages

Pass	Message	Type	NLP vars	Result interval
collect	$\mu(S_X \rightarrow X)$	F $\rightarrow$ V (LP)	1	[0.6000, 0.8000]
collect	$\mu(S_Y \rightarrow Y)$	F $\rightarrow$ V (LP)	1	[0.3000, 0.5000]
collect	$\mu(X \rightarrow S_A)$	V $\rightarrow$ F	–	[0.6000, 0.8000]
collect	$\mu(Y \rightarrow S_{BCD})$	V $\rightarrow$ F	–	[0.3000, 0.5000]
collect	$\mu(S_A \rightarrow A)$	F $\rightarrow$ V (NLP)	4	[0.4200, 0.9400]
collect	$\mu(A \rightarrow S_{BCD})$	V $\rightarrow$ F	–	[0.4200, 0.9400]
collect	$\mu(S_{BCD} \rightarrow C)$	F $\rightarrow$ V (NLP)	32	[0.1260, 0.9580]
collect	$\mu(C \rightarrow S_E)$	V $\rightarrow$ F	–	[0.1260, 0.9580]
distributed	$\mu(S_E \rightarrow C)$	F $\rightarrow$ V (NLP)	4	[0.3333, 0.8571]
distributed	$\mu(C \rightarrow S_{BCD})$	V $\rightarrow$ F	–	[0.3333, 0.8571]
distributed	$\mu(S_{BCD} \rightarrow A)$	F $\rightarrow$ V (NLP)	32	[0.0000, 1.0000] (vac.)
distributed	$\mu(S_{BCD} \rightarrow Y)$	F $\rightarrow$ V (NLP)	32	[0.0000, 1.0000] (vac.)
distributed	$\mu(A \rightarrow S_A)$	V $\rightarrow$ F	–	[0.0000, 1.0000] (vac.)
distributed	$\mu(S_A \rightarrow X)$	F $\rightarrow$ V (NLP)	4	[0.0000, 1.0000] (vac.)
distributed	$\mu(Y \rightarrow S_Y)$	V $\rightarrow$ F	–	[0.0000, 1.0000] (vac.)
extract	$\mathcal{K}(X)$	intersect	–	[0.6000, 0.8000]
extract	$\mathcal{K}(A)$	intersect	–	[0.4200, 0.9400]
extract	$\mathcal{K}(Y)$	intersect	–	[0.3000, 0.5000]
extract	$\mathcal{K}(C)$	intersect	–	[0.3333, 0.8571]
extract	$\mathcal{K}(B), \mathcal{K}(D)$	F final (NLP)	32	[0.2100, 0.8200], [0.1333, 0.8667]
extract	$\mathcal{K}(E)$	F final (NLP)	4	[0.5000, 0.7000]

5.8 What the example demonstrates

- The cycle $B \rightarrow C \rightarrow D \rightarrow B$ is *never* traversed by a message; it is dissolved inside the single 32-variable NLP of $[S_{BCD}]$ and solved jointly.
- Every inter-super-node message is an interval on *one* interface variable, computed by a clearly defined min/max objective.
- Multi-parent handling: $[S_{BCD}]$ correctly combines the two incoming messages from A and Y in each of its outgoing computations, excluding the destination’s own message (the generalized BP rule).
- Final beliefs at interfaces are pure interval intersections; final beliefs at hidden atoms B, D need one extra local NLP.
- The forward pass alone already produces tight intervals here; the backward pass tightens *only* $\mathcal{K}(C)$ (from [0.126, 0.958] to [0.333, 0.857]), driven by the leaf prior c_E through $[S_E]$. All other backward messages are vacuous because this network’s dependencies point strictly downstream.
- Every reported interval is an actual optimum, and the final marginals coincide *exactly* with the 128-world global NLP (Table 1): SCC-FGP is exact on this polytree-condensation instance.

6 Comparison with the ARIEL Scheme

ARIEL (Marinescu et al., *Approximate Inference in LCNs*, IJCAI-2023) is the existing approximate message-passing algorithm for LCNs. Both ARIEL and SCC-FGP use factor graphs and pass interval (credal-set) messages, but they differ fundamentally in *which* graph they build and in *how* they treat cycles.

6.1 Critical Difference 1 — the graph representation

- **ARIEL** builds its factor graph *directly* from the original LCN: variable nodes are the individual atoms (A, B, C, D, E, X, Y) , and factor nodes are the individual constraints (one factor per sentence $c_X, c_{XA}, c_{AB}, c_{BC}, \dots$). Because constraints share atoms, the graph is typically **cyclic** with many short and long loops.
- **SCC-FGP** builds its factor graph on the *condensation* DAG: variable nodes are only the *interface* atoms (X, A, Y, C, E) , and factor nodes are the *super-nodes* $([S_X], [S_A], [S_{BCD}], [S_E], [S_Y])$, each of which packages an entire sub-LCN. The resulting graph is **guaranteed acyclic**.

6.2 Critical Difference 2 — handling of cycles

- **ARIEL** runs *loopy* belief propagation: it iterates messages around the cyclic graph until a fixed point (or a max-iter cap), *hoping* for convergence. Approximation error comes from evidence “double-counting” as messages loop back on themselves; on the example, the loop $B \rightarrow C \rightarrow D \rightarrow B$ is traversed repeatedly.
- **SCC-FGP** *isolates* every cycle inside a super-node. Propagation runs on the acyclic graph, so it is *finite* (two passes) and *exact* at the global level—there is no loop to double-count over.

6.3 Critical Difference 3 — the NLPs

- **ARIEL** solves *many small, local* NLPs: to compute the message out of the factor for $c_{BC} : P(C \mid B) \in [0.60, 0.80]$, the NLP involves only B, C and the incoming messages on them. Many such NLPs are solved *per iteration*, over many iterations.
- **SCC-FGP** solves *fewer but larger* NLPs: the message $\mu(S_{BCD} \rightarrow C)$ requires the full 32-variable joint over $\{B, C, D, A, Y\}$, incorporating multiple incoming messages at once. There are $O(|E(\mathcal{F})|)$ NLPs total (no iteration).

6.4 Side-by-side trace on the example

Consider the cyclic part $\{B, C, D\}$ with c_{BC}, c_{CD}, c_{DB} .

ARIEL’s view. Three separate factors $[c_{BC}], [c_{CD}], [c_{DB}]$ and three variable nodes $(B), (C), (D)$ form a triangle:

$$(B) - [c_{BC}] - (C) - [c_{CD}] - (D) - [c_{DB}] - (B).$$

This is a *loop*. ARIEL initializes all messages to $[0, 1]$ and iterates: each factor recomputes its outgoing intervals from the incoming intervals on its two atoms by solving a 2-variable local NLP, e.g.

$$\mu([c_{BC}] \rightarrow C) : \quad \min / \max P(C=1) \text{ s.t. } 0.60 \leq P(C \mid B) \leq 0.80, \quad \mu(B \rightarrow [c_{BC}]).$$

The interval on C feeds $[c_{CD}]$, whose output on D feeds $[c_{DB}]$, whose output on B feeds back into $[c_{BC}]$ —and the cycle repeats until the intervals stop shrinking (or oscillate). Each loop re-uses information that has already circulated, which is why the fixed-point intervals are an *outer* (loose) approximation. Because each 2-variable NLP sees only one conditional and never the joint coupling of all three, ARIEL cannot reproduce the joint feasible region of $\{B, C, D\}$; its fixed-point interval for C is a *superset* of the exact $[0.126, 0.958]$ that $[S_{BCD}]$ computes in one shot.

SCC-FGP’s view. The same three constraints are *all* owned by the single factor $[S_{BCD}]$. There is *no* triangle and *no* iteration: a single NLP over the $2^3 = 8$ (here 32 with interfaces) joint worlds enforces c_{BC}, c_{CD}, c_{DB} *simultaneously*, so the mutual dependence among B, C, D is captured exactly — yielding the forward message $\mu(S_{BCD} \rightarrow C) = [0.1260, 0.9580]$ from step (1.6), which (after the backward pass) is part of the exact final marginal $\mathcal{K}(C) = [0.3333, 0.8571]$. The interval on C that leaves $[S_{BCD}]$ is therefore at least as tight as—and generally tighter than—ARIEL’s looped fixed point.

6.5 Summary table

Feature	ARIEL	SCC-FGP
Graph used	original LCN factor graph	condensation factor graph
Variable nodes	all atoms	interface atoms only
Factor nodes	individual constraints	super-nodes (SCCs)
Graph structure	potentially cyclic	guaranteed acyclic (DAG)
Cycle handling	implicit (loopy BP)	explicit (cycles inside factors)
Global propagation	iterative, heuristic, may not converge	finite, exact, 2 passes
Source of approx.	looping messages double-count evidence	only the local NLP inside a large SCC
NLP scope	small & local (per constraint)	large & complex (per super-node)
# NLPs	many, $\times$ iterations	few ($O(E(\mathcal{F}))$), no iteration
Pre-processing	minimal (build graph)	significant (structure graph, Tarjan, condensation)

6.6 A hybrid relationship

SCC-FGP is best understood as a *meta-algorithm* that *wraps* ARIEL rather than replacing it. The large NLP inside a cyclic super-node is itself an inference problem on a cyclic sub-LCN. When that NLP is too large for an exact solver, the natural subroutine to solve it approximately is **ARIEL** (or another credal loopy-BP). Thus:

- SCC-FGP provides a *globally structured, exact* way to propagate beliefs across the acyclic part of the network;
- it confines any *loopy* approximation to the irreducible cyclic cores it has isolated, preventing loop-induced error from “polluting” the rest of the inference.

In the extreme cases, SCC-FGP with all-singleton SCCs and exact local solves on an acyclic LCN reduces to exact two-pass propagation, whereas a single SCC containing the whole network reduces to the global NLP of Section 1—ARIEL sits in between, never isolating cycles at all.

7 Complexity and Practical Notes

- **Decomposition** is $O(|V| + |E|)$ via Tarjan plus the cheap condensation and factor-graph build.
- **Propagation** computes $O(|E(\mathcal{F})|)$ messages in two passes; no iteration to convergence.
- **Local NLP size** for super-node S_j is 2^{m_j} variables, where $m_j = |\text{Vars}(S_j)| + |\text{Int}(S_j)|$. This is the dominant cost and the *only* place exponential blow-up can occur—bounded by the size of the largest SCC plus its interface, *not* by n .
- **Tightness vs. the global NLP.** SCC-FGP loses some tightness only through the marginalization at super-node boundaries (a message summarizes a super-node by an interval on one interface atom). It is provably at least as tight as ARIEL on any network whose cycles it isolates, and exact whenever every SCC is solved exactly and the condensation graph is a polytree.

Open issues (from the ideation). (i) *Scalability of local solvers* for very large SCCs—research into scalable LP/NLP or approximate (ARIEL) subroutines for the inner problem; (ii) *Richer messages*: passing the full marginal credal polytope $\mathcal{K}(\mathbf{X}_p)$ over a multi-atom interface (via Fourier–Motzkin) rather than per-atom intervals, trading representation cost for additional tightness.

References

1. H. Qian, R. Marinescu, A. Gray, et al. Logical Credal Networks. *NeurIPS*, 2021.
2. R. Marinescu, H. Qian, A. Gray, et al. Approximate Inference in Logical Credal Networks. *IJCAI*, 2023.
3. R. Tarjan. Depth-first search and linear graph algorithms. *SIAM J. Computing*, 1(2):146–160, 1972.
4. A. Antonucci, C.P. de Campos, D. Huber, M. Zaffalon. Approximate credal network updating by linear programming with applications to decision making. *Int. J. Approx. Reasoning*, 58:25–38, 2015.
5. A. Charnes, W.W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics*, 9:181–186, 1962.
6. D.D. Mauá, F.G. Cozman. Thirty years of credal networks. *Int. J. Approx. Reasoning*, 126:133–157, 2020.